大模型核心技术深度解析

从注意力算子到后训练对齐

FlashAttention • MoE • 量化 • 稀疏 • 投机解码 • RLHF • DPO • KV Cache

推理优化 • 模型压缩 • 偏好学习 • 能力涌现 • 工程部署

2026 年 5 月 19 日

目录

第一章 注意力机制深度解析 **7**

1.1 标准自注意力（回顾） 7

1.1.1 因果掩码 7

1.2 FlashAttention 8

1.2.1 核心动机 8

1.2.2 在线 Softmax（Online Softmax） 8

1.2.3 FlashAttention 分块算法 9

1.2.4 FlashAttention v2/v3 改进 9

1.3 PagedAttention (vLLM) 10

1.3.1 KV Cache 管理问题 10

1.3.2 PagedAttention 核心思想 11

1.3.3 PagedAttention 的注意力计算 11

1.4 注意力算子底层实现 12

1.4.1 CUDA Kernel 设计要点 12

第二章 模型架构创新 **15**

2.1 MoE（混合专家） 15

2.1.1 核心原理 15

2.1.2 负载均衡 15

2.1.3 关键架构变体 16

2.2 稀疏注意力机制 17

2.2.1 Sliding Window Attention (SWA) 17

2.2.2 Dilated Sliding Window (DSA / Longformer) 17

2.2.3 其他稀疏注意力 18

2.3 模型融合（Model Merging） 18

第三章 推理优化 **19**

3.1 KV Cache 管理与复用 19

3.1.1 量化 KV Cache 19

3.1.2 KV Cache 复用 19

3.1.3 多轮对话中的 KV Cache . . . 19
3.2 Chunked Prefill . . . 20
3.3 投机解码（Speculative Decoding） . . . 21
3.3.1 核心思想 . . . 21
3.3.2 接受/拒绝采样 . . . 21
3.3.3 Medusa 与 Eagle . . . 21
3.4 分离式部署（PD: Prefill-Decode Disaggregation） . . . 22
3.5 大模型通信算子 . . . 23
3.5.1 张量并行（Tensor Parallelism, TP） . . . 23
3.5.2 流水线并行（Pipeline Parallelism, PP） . . . 23
3.5.3 数据并行（Data Parallelism, DP） . . . 24
3.5.4 序列并行（Sequence Parallelism, SP） . . . 24

第四章 模型量化 27
4.1 量化基础 . . . 27
4.2 GPTQ . . . 27
4.3 AWQ . . . 28
4.4 量化对比与面试要点 . . . 29

第五章 模型压缩与蒸馏 31
5.1 稀疏化 . . . 31
5.1.1 非结构化稀疏 . . . 31
5.1.2 2:4 结构化稀疏 . . . 31
5.1.3 半结构化稀疏（SparseGPT, Wanda） . . . 31
5.2 蒸馏技术 . . . 32
5.2.1 知识蒸馏（Knowledge Distillation） . . . 32
5.2.2 LLM 特定蒸馏 . . . 32
5.3 多任务强化学习（MTRL） . . . 33

第六章 训练与对齐 35
6.1 SFT（监督微调） . . . 35
6.2 RLHF（基于人类反馈的强化学习） . . . 36
6.2.1 三阶段流程 . . . 36
6.2.2 奖励模型训练 . . . 36
6.2.3 PPO 微调 . . . 36
6.3 偏好学习（DPO 等） . . . 37
6.3.1 DPO（Direct Preference Optimization） . . . 37
6.3.2 DPO 变体 . . . 38
6.4 模型对齐全景 . . . 39

第七章 能力专题 **41**
7.1 Planning（规划） 41
7.2 Reasoning（推理） 41
7.3 Coding（代码生成） 42
7.4 Instruction Following 43
7.5 Hallucination Control 44
7.6 Context Learning（上下文学习） 45

第八章 系统与工程 **47**
8.1 MFU（模型浮点运算利用率） 47
8.2 后训练优化（Post-Training） 48
8.3 总结：大模型技术栈全景图 48

第一章　注意力机制深度解析

1.1　标准自注意力（回顾）

缩放点积注意力

给定查询 $\mathbf{Q} \in \mathbb{R}^{n \times d_k}$，键 $\mathbf{K} \in \mathbb{R}^{n \times d_k}$，值 $\mathbf{V} \in \mathbb{R}^{n \times d_v}$：

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V} \tag{1.1}$$

计算步骤：

1. $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{n \times n}$ ——$\mathrm{O}(n^2 d)$ 计算，$\mathrm{O}(n^2)$ 显存
2. $\mathbf{P} = \text{softmax}(\mathbf{S}/\sqrt{d_k})$ ——逐行 softmax
3. $\mathbf{O} = \mathbf{P}\mathbf{V}$ ——$\mathrm{O}(n^2 d)$ 计算

多头注意力：将 d_{model} 拆分为 h 个头，每个头独立计算，最后拼接：

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\mathbf{W}^O \tag{1.2}$$

其中 $\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V)$。

1.1.1　因果掩码

在自回归（decoder-only）模型中，token i 只能关注 token $j \le i$。通过将 $\mathbf{S}$ 的上三角部分设为 $-\infty$ 实现：

$$\mathbf{S}_{ij}^{\text{masked}} = \begin{cases} \mathbf{S}_{ij}, & j \le i \\ -\infty, & j > i \end{cases} \tag{1.3}$$

1.2 FlashAttention

1.2.1 核心动机

标准注意力的瓶颈在**显存**而非计算：$\mathbf{S} \in \mathbb{R}^{n\times n}$ 需要 $\mathrm{O}(n^2)$ 显存。当 $n = 8192$ 时，$\mathbf{S}$ 约 256MB（fp32）。当 $n = 131072$（128K 上下文），$\mathbf{S}$ 约 64GB——远超 GPU 显存。

FlashAttention 的核心观察：**softmax 的增量计算特性**允许分块计算，避免实例化完整的 $\mathbf{S}$ 矩阵。

FlashAttention 关键洞察

显存瓶颈：GPU 的 SRAM（ 20MB）和 HBM（ 80GB）之间带宽差距极大（ 12.8×）。标准注意力反复读写 HBM。

解决方案：在 SRAM 内完成分块计算，每次只加载一块 $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ 到 SRAM，计算该块的注意力，用**在线 softmax** 更新最终输出。

I/O 复杂度：从 $\mathrm{O}(n^2)$ 降为 $\mathrm{O}(n^2 d/M^{1/2})$，其中 M 是 SRAM 大小。实际加速 2–8×。

1.2.2 在线 Softmax（Online Softmax）

标准 softmax 需要三次遍历数组（求 max、求 exp 和、归一化）。在线 softmax 仅需一次遍历。

Algorithm 1 Online Softmax

输入： 向量 $\mathbf{x} = [x_1, \ldots, x_n]$

$m_0 \leftarrow -\infty,\ \ell_0 \leftarrow 0$
for $i = 1$ **to** n **do**
  $m_i \leftarrow \max(m_{i-1}, x_i)$
  $\ell_i \leftarrow \ell_{i-1} \cdot e^{m_{i-1}-m_i} + e^{x_i - m_i}$
for $i = 1$ **to** n **do**
  $y_i \leftarrow e^{x_i - m_n}/\ell_n$
return y

证明（面试重点）： 假设已有 $m = \max(x_1, \ldots, x_k)$ 和 $\ell = \sum_{j=1}^{k} e^{x_j - m}$。加入新元素 x_{k+1} 后：

若 $x_{k+1} \le m$（最大值不变）：

$$\ell_{\text{new}} = \ell + e^{x_{k+1}-m} \tag{1.4}$$

若 $x_{k+1} > m$（最大值更新）：

$$m_{\text{new}} = x_{k+1}, \quad \ell_{\text{new}} = \ell \cdot e^{m-m_{\text{new}}} + 1 \tag{1.5}$$

关键在于 $\ell \cdot e^{m-m_{\text{new}}}$ 项将所有旧项的指数重新缩放到新的最大值下。

1.2.3　FlashAttention 分块算法

算法：FlashAttention 前向传播

Algorithm 2 FlashAttention Forward (简化版)

输入： $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{n \times d}$, 块大小 B_r, B_c

将 $\mathbf{Q}$ 分成 $T_r = \lceil n/B_r \rceil$ 块, 将 $\mathbf{K}, \mathbf{V}$ 分成 $T_c = \lceil n/B_c \rceil$ 块
将 $\mathbf{O}$ 初始化为 (n, d) 全零矩阵
将 ℓ 初始化为 $(n,)$ 全零向量 (softmax 分母)
将 m 初始化为 $(n,)$ 全 $-\infty$ 向量 (softmax 最大值)
for $i = 1$ **to** T_r **do**
    加载 $\mathbf{Q}_i \in \mathbb{R}^{B_r \times d}$ 到 SRAM
    for $j = 1$ **to** T_c **do**
        加载 $\mathbf{K}_j, \mathbf{V}_j \in \mathbb{R}^{B_c \times d}$ 到 SRAM
        $\mathbf{S}_{ij} \leftarrow \mathbf{Q}_i \mathbf{K}_j^\top \in \mathbb{R}^{B_r \times B_c}$
        $m_{ij} \leftarrow \text{rowmax}(\mathbf{S}_{ij})$ ▷ 局部最大值
        $\mathbf{P}_{ij} \leftarrow \exp(\mathbf{S}_{ij} - m_{ij})$ ▷ 局部 *softmax* 分子
        $\ell_{ij} \leftarrow \text{rowsum}(\mathbf{P}_{ij})$ ▷ 局部 *softmax* 分母
        $m_{\text{new}} \leftarrow \max(m_i, m_{ij})$ ▷ 更新全局最大值
        $\ell_{\text{new}} \leftarrow \ell_i \cdot e^{m_i - m_{\text{new}}} + \ell_{ij} \cdot e^{m_{ij} - m_{\text{new}}}$
        $\mathbf{O}_i \leftarrow \text{diag}(e^{m_i - m_{\text{new}}}) \cdot \mathbf{O}_i + e^{m_{ij} - m_{\text{new}}} \cdot \mathbf{P}_{ij} \mathbf{V}_j$
        $m_i \leftarrow m_{\text{new}}, \ell_i \leftarrow \ell_{\text{new}}$
    $\mathbf{O}_i \leftarrow \text{diag}(1/\ell_i) \cdot \mathbf{O}_i$ ▷ 最终归一化
    写回 $\mathbf{O}_i$ 到 HBM
return $\mathbf{O}$

1.2.4　FlashAttention v2/v3 改进

FA-v2 改进

- **减少非矩阵乘法操作：**优化 softmax 重缩放，减少逐元素操作
- **并行化序列长度维度：**v1 在 batch 和 head 维度并行，v2 额外在序列长度维度并行
- **改善 warp 调度：**更好的 SM 占用率
- 相比 v1 约 2× 加速，达到理论峰值的 70%+

FA-v3 (Hopper GPU)

专为 H100/H200 架构优化：

- **异步执行**：利用 Hopper 的 TMA（Tensor Memory Accelerator）和 WGMMA 指令
- **FP8 支持**：原生支持 FP8 计算，进一步减少显存
- 在 H100 上可达 740 TFLOPS（理论峰值的 75%）

面试常见题

Q: FlashAttention 为什么能加速？
A: 核心是减少了 HBM 读写。标准注意力的 **S** 矩阵（$n \times n$）必须在 HBM 中实例化，读写量为 $O(n^2)$。FlashAttention 使用分块 +tiling，在 SRAM 中完成 softmax 增量计算，**S** 从不完整写入 HBM。I/O 复杂度从 $O(n^2)$ 降为 $O(n^2d^2/M)$。
Q: 为什么在线 softmax 是数值稳定的？
A: 标准 softmax 需要先减去最大值防止溢出（$\exp(x_i - \max)$）。在线版本维护一个运行中的最大值 m。当发现更大的值时，用 $\exp(m_{\text{old}} - m_{\text{new}})$ 将之前的所有部分和重新缩放。这等价于一次性计算。
Q: FlashAttention 对哪些场景加速最大？
A: 长序列场景（预填充阶段）。对于 decode 阶段（逐 token 生成，$n = 1$），FlashAttention 优势不如预填充显著，因为 **S** 矩阵本身是 $1 \times n$，显存已很小。

1.3 PagedAttention (vLLM)

1.3.1 KV Cache 管理问题

自回归推理中，每生成一个新 token，需要计算该 token 对所有历史 token 的注意力。每次重新计算所有历史 KV 极其低效，因此缓存历史 KV——这就是 **KV Cache**。

KV Cache

对每个 transformer 层，缓存所有历史 token 的 $\mathbf{K}, \mathbf{V}$。生成第 t 个 token 时：

$$\mathbf{q}_t = \mathbf{x}_t \mathbf{W}^Q, \quad \mathbf{K}_{[1:t]} = \text{Concat}(\mathbf{K}_{[1:t-1]}, \mathbf{x}_t \mathbf{W}^K), \quad \mathbf{V}_{[1:t]} = \text{Concat}(\mathbf{V}_{[1:t-1]}, \mathbf{x}_t \mathbf{W}^V) \tag{1.6}$$

$$\mathbf{o}_t = \text{softmax}(\mathbf{q}_t \mathbf{K}_{[1:t]}^\top / \sqrt{d}) \mathbf{V}_{[1:t]} \tag{1.7}$$

KV Cache 总大小 $= 2 \times n_{\text{layers}} \times n_{\text{heads}} \times d_{\text{head}} \times \text{seq_len} \times \text{dtype_bytes}$
例如 Llama-2-70B：$2 \times 80 \times 64 \times 128 \times 4096 \times 2 = 10\text{GB}$ per 4K tokens。

标准 KV Cache 管理（如 HuggingFace）使用**预分配连续显存**，造成：

- **显存碎片化**：不同请求的序列长度不同，预分配 = 大量浪费（内部碎片）
- **无法共享**：并行采样（beam search）或前缀共享时，无法复用公共前缀的 KV

1.3.2　PagedAttention 核心思想

PagedAttention

借鉴操作系统的分页机制：

1. KV Cache 被分割为固定大小的**块**（block，如 16 或 32 个 token）
2. 块可以放在 GPU 显存的任意非连续位置（类似物理页框）
3. 维护一个块表（block table）记录每个请求的逻辑块到物理块的映射

类比：

- 逻辑块 : 物理块 = 虚拟页 : 物理页框
- 块表 = 页表

PagedAttention 关键优势

1. **零碎片浪费**：按需分配块，不使用预分配。利用率从 20% ∼ 40% 提升到 96%+。
2. **灵活共享**：
   - **前缀共享**：多个请求共享相同前缀时（如系统提示词），物理块可以共享，引用计数 > 1
   - **Copy-on-Write**：只有当某请求修改共享块时才复制
   - **并行采样**：beam search 或温度采样生成多个候选时，共享 prompt 的 KV
3. **显存效率**：vLLM 的吞吐量是 HuggingFace Transformers 的 14× ∼ 24×。

1.3.3　PagedAttention 的注意力计算

PagedAttention Kernel

1. 对 query token $\mathbf{q}$，遍历块表中该请求的所有逻辑块
2. 对每个逻辑块，通过块表查找物理块地址
3. 加载该物理块的 $\mathbf{K}, \mathbf{V}$ 数据
4. 计算该块的注意力分数，用在线 softmax 累加

5. 处理完所有块后，最终归一化并输出

面试常见题

Q: PagedAttention 和 FlashAttention 是什么关系？

A: 正交技术。FlashAttention 是**计算优化**（tiling + kernel fusion），解决单次注意力的显存和速度问题。PagedAttention 是**显存管理优化**，解决多条请求下 KV Cache 的碎片和共享问题。两者结合使用：vLLM 使用 PagedAttention 管理 KV 块布局，实际计算调用 FlashAttention kernel。

Q: KV Cache 为何成为推理瓶颈？

A: 对于 Llama-70B，生成长度为 4096 时，KV Cache 约 10GB。8 个并发请求就需要 80GB——这远超 A100 的显存。此外，KV Cache 的 I/O 带宽受限：每次生成新 token 需要读取所有历史 KV，对显存带宽的压力是计算瓶颈的前置因素。这就是为什么大部分 LLM 推理是**显存带宽受限**而非算力受限。

1.4 注意力算子底层实现

1.4.1 CUDA Kernel 设计要点

高性能注意力 Kernel 的关键

1. **线程块（Thread Block）划分：**
   - 每个 thread block 负责 $\mathbf{Q}$ 的一个或多个块
   - 循环遍历 $\mathbf{K}, \mathbf{V}$ 的块
   - Block size 的选择要平衡 SM 占用率和 SRAM 使用
2. **共享内存（Shared Memory）布局：**
   - $\mathbf{Q}$ tile, $\mathbf{K}$ tile, $\mathbf{V}$ tile 必须驻留在 shared memory
   - 52KB–100KB shared memory per SM（依赖 GPU 型号）
   - Bank conflict 优化：$\mathbf{Q}$ 和 $\mathbf{K}$ 的加载维度要注意对齐
3. **寄存器使用：**
   - 每线程维护局部 softmax 状态（m, ℓ）在寄存器中
   - 减少 shared memory 和寄存器的溢出
4. **内存合并（Coalesced Access）：**
   - HBM 读写要按 warp 对齐（128 字节对齐）

- $\mathbf{K}, \mathbf{V}$ 的加载方向要匹配内存布局

5. **异步拷贝：**
   - 使用 'cp.async'（SM80+）或 TMA（SM90+）做异步 HBM→SRAM 拷贝
   - 拷贝和数据计算重叠（双缓冲）
6. **Warp 级别优化：**
   - 使用 warp shuffle 做 warp 内 reduction
   - MMA（Matrix Multiply-Accumulate）指令利用 Tensor Core

面试常见题：手写简化版 FlashAttention

Q: 请用伪代码写出 FlashAttention 的核心循环。

（见上文 FlashAttention 伪代码）

关键追问：

- **为什么 S 矩阵不需要完整存储？**因为 softmax 的每行是独立的，第 i 行的归一化只依赖于第 i 行的值。
- **重缩放因子 $e^{m_i - m_{\text{new}}}$ 怎么推导的？**从 softmax 定义出发，分子分母同除 $e^{m_{\text{new}}}$，旧的分母 $\ell_{\text{old}} = \sum \exp(x_j - m_{\text{old}})$，除以 $e^{m_{\text{new}} - m_{\text{old}}}$ 后 $= \sum \exp(x_j - m_{\text{new}}) = \ell_{\text{old}} \cdot e^{m_{\text{old}} - m_{\text{new}}}$。
- **如何处理因果掩码？**对行 i，只遍历 $j \le i$ 的 KV 块。在加载 $\mathbf{K}_j$ 块前检查 j 块是否在因果范围内。

第二章　模型架构创新

2.1　MoE（混合专家）

2.1.1　核心原理

MoE 将 FFN 层替换为多个并行的" 专家" 子网络，由一个门控网络动态决定每个 token 激活哪些专家。

MoE 公式

给定输入 $\mathbf{x} \in \mathbb{R}^d$，MoE 层的输出为：

$$\text{MoE}(\mathbf{x}) = \sum_{i=1}^{N} G(\mathbf{x})_i \cdot E_i(\mathbf{x}) \tag{2.1}$$

其中：

- N：专家总数（如 Mixtral 8x7B 中的 8）
- E_i：第 i 个专家（通常是一个 FFN）
- $G(\mathbf{x})$：门控网络，输出每个专家的权重

稀疏 MoE（实际使用的形式）：仅激活 top-k 个专家（$k \ll N$，通常 $k = 2$）：

$$G(\mathbf{x}) = \text{softmax}(\text{TopK}(\mathbf{x} \cdot \mathbf{W}_g, k)) \tag{2.2}$$

其中 TopK 保留最大的 k 个 logit，其余置为 $-\infty$（softmax 后为 0）。

2.1.2　负载均衡

负载均衡损失

MoE 的经典问题是**专家坍塌**：门控网络倾向于总选同一批专家，其他专家退化。解决方法是添加辅助的负载均衡损失：

$$\mathcal{L}_{\text{aux}} = \alpha \cdot N \cdot \sum_{i=1}^{N} f_i \cdot P_i \tag{2.3}$$

其中：

- f_i：分配给专家 i 的 token 比例
- P_i：门控网络分配给专家 i 的 softmax 概率均值
- α：超参数（如 0.01）

当 f_i 和 P_i 都均匀分布时（$= 1/N$），$\mathcal{L}_{\text{aux}}$ 最小。

2.1.3 关键架构变体

Switch Transformer 与 Mixtral

Switch Transformer (Google, 2021)：

- Top-1 路由（每个 token 只选一个专家）
- 容量因子：限制每个专家能处理的最大 token 数，超出的 token 被丢弃（token dropping）
- 在相同计算量下提升训练速度 7×

Mixtral 8x7B (Mistral, 2024)：

- 8 个专家，每个 token 激活 Top-2
- 总参数量 47B，但每个 token 只通过 13B 活跃参数
- 推理速度 = 13B dense 模型的速度
- 性能接近/超过 Llama-2-70B

DeepSeekMoE：

- 细粒度专家划分（更多更小的专家，如 64 个）
- 共享专家 + 路由专家的混合
- 每个 token 激活 top-6 路由专家 + 2 个共享专家

面试常见题

Q: MoE 的参数量和计算量是什么关系？

A: 总参数量 $= N \times$ (单专家参数量)，但每个 token 只激活 k 个专家。活跃参数量 $= k \times$ (单专家参数量)。FLOPs 正比于活跃参数量，而非总参数量。这就是为什么 Mixtral 8x7B（47B 参数）的推理速度和 13B dense 模型相当。

Q: MoE 推理的显存需求？

A: 所有专家参数都要加载到显存（即使不激活），所以 MoE 的显存需求 = 总参数量，而不是活跃参数量。这使得 MoE 的推理需要更多 GPU。

Q: 专家坍塌的本质原因？

A: 门控网络的梯度更新倾向于" 富者愈富"——早期表现好的专家获得更多 token，进而获得更多训练信号，变得更擅长，形成正反馈。负载均衡损失是一个正则化项，强制 token 分布均匀。

2.2　稀疏注意力机制

2.2.1　Sliding Window Attention (SWA)

Sliding Window Attention

Mistral 的核心设计。每个 token 只关注其局部窗口内的 w 个 token（如 $w = 4096$）：

$$\text{Attention}_{\text{SWA}}(\mathbf{Q}, \mathbf{K}, \mathbf{V})_{ij} = \begin{cases} \text{softmax}(\mathbf{q}_i^\top \mathbf{k}_j / \sqrt{d}), & j \in [i-w, i] \\ 0, & \text{otherwise} \end{cases} \tag{2.4}$$

复杂度： $O(n \cdot w)$ 而非 $O(n^2)$。对于 128K 序列，$w = 4K$ 时复杂度降 32×。

全局信息流： 信息通过多层堆叠传播。第 L 层能" 看到" $L \times w$ 范围内的信息（感受野）。

2.2.2　Dilated Sliding Window (DSA / Longformer)

空洞滑动窗口

Longformer 提出，在滑动窗口基础上增加空洞模式：

- **滑动窗口：** 关注局部邻居，窗口大小 w
- **空洞窗口：** 以间隔 d 采样，关注 $[i-w, i]$ 中每隔 d 步的 token
- **全局注意力：** 少数 token（如 [CLS], 分隔符）关注所有 token

空洞设计增加了感受野，一层就能覆盖 $(w/d) \times (w)$ 的范围。多层堆叠效果更显著。

2.2.3 其他稀疏注意力

稀疏注意力谱系

- **Sparse Transformer (OpenAI)**：固定的稀疏模式（strided + local），训练时就确定
- **Reformer**：基于 LSH（局部敏感哈希）的稀疏注意力，只关注哈希值接近的 token
- **BigBird (Google)**：random + local + global 三种注意力模式组合
- **Mamba / State Space Models**：完全不同于注意力的序列建模方式，复杂度 $O(n)$

2.3 模型融合（Model Merging）

模型融合技术

将多个 fine-tuned 模型合并为一个，无需额外训练。常用方法：

1. 线性插值（Linear / LERP）：

$$\theta_{\text{merged}} = (1-\lambda)\theta_A + \lambda\theta_B \tag{2.5}$$

2. SLERP（球面线性插值）：在权重空间做球面插值，保持权重范数：

$$\theta_{\text{merged}} = \frac{\sin((1-\lambda)\Omega)}{\sin\Omega}\theta_A + \frac{\sin(\lambda\Omega)}{\sin\Omega}\theta_B \tag{2.6}$$

其中 Ω 是 θ_A 和 θ_B 之间的夹角。

3. TIES-Merging：三步流程：

1. **Trim**：只保留变化显著的参数（与基础模型的差值的 top-k%）
2. **Elect Sign**：对多个模型的同位置参数，投票选举最终符号
3. **Disjoint Merge**：只合并符号一致的参数

4. DARE：Drop And REscale：随机丢弃 delta 参数（置零）后重新缩放，再合并。减少任务间干扰。

5. Task Arithmetic：$\theta_{\text{merged}} = \theta_{\text{base}} + \lambda\sum_i(\theta_i - \theta_{\text{base}})$ 将多个任务向量的和加到基础模型上，配合 TIES 或 DARE 效果更好。

第三章　推理优化

3.1　KV Cache 管理与复用

3.1.1　量化 KV Cache

KV Cache 可以量化为 INT8 或 FP8 以减少显存：

$$\tilde{\mathbf{K}} = \text{quantize}(\mathbf{K}, \text{scale}_K, \text{zero}_K) \tag{3.1}$$

常见方案：

- **KIVI**：对 **K** 按 channel 量化，**V** 按 token 量化（因为 **K** 和 **V** 的分布差异大）
- **FlexGen**：将 KV Cache offload 到 CPU 内存和磁盘，结合量化进一步压缩

3.1.2　KV Cache 复用

前缀缓存（Prefix Caching）

多个请求共享相同前缀（如系统提示词 System Prompt），前缀的 KV Cache 计算一次，所有后续请求复用。

vLLM 实现：使用哈希表将前缀 token 序列映射到物理块。新请求来时先查哈希表，命中则共享物理块（引用计数 +1）。

Hydragen：将注意力分解为"prefix attention" 和"suffix attention" 两部分，prefix 部分可以批量高效计算。

3.1.3　多轮对话中的 KV Cache

多轮对话缓存策略

每轮对话追加新消息，旧消息的 KV 不变。策略：

1. **全量保留**：保留所有轮的 KV Cache，新轮只追加。显存随轮数线性增长。
2. **滑动窗口**：只保留最近 k 轮的 KV Cache。
3. **语义压缩**：用小型模型将历史轮次的 KV 压缩为更紧凑的表示。

4. **Layer-wise 复用**：前几层的 KV Cache 变化小，可以跨轮共享。

面试常见题

Q: 为什么 LLM 推理是 memory-bound 而非 compute-bound?
A: 生成阶段（decode），每次只处理 1 个新 token。计算量 $O(d^2 + nd)$（n 是序列长度），而显存读写量 $O(nd)$。当 n 很大时（如 8K+），读取 KV Cache 的 I/O 时间远超矩阵乘法时间。因此 decode 阶段受限于 HBM 带宽。
Q: 如何估计一个模型需要的 KV Cache 显存?
A: KV_size $= 2 \times L \times H \times D \times S \times$ bytes，其中 L= 层数, H= 头数, D= 每头维度, S= 序列长度, bytes=fp16=2 或 fp8=1。例如 Llama-2-7B：$2 \times 32 \times 32 \times 128 \times 2048 \times 2 \approx 1$GB。

3.2 Chunked Prefill

Chunked Prefill（分块预填充）

预填充（Prefill）处理输入 prompt 的所有 token，一次前向计算生成第一个输出 token。**Chunked Prefill** 将长 prompt 切成多个 chunk（如每块 512 tokens），逐个处理：
动机：

1. 长 prompt 预填充可能导致 GPU 显存溢出（OOM）
2. 预填充耗时过长会导致其他请求的 decode 被阻塞
3. 通过分块，可以在 chunk 之间插入 decode 请求，提高并发

Sarathi-Serve / vLLM 的实现：

- 将预填充预算（prefill budget）限制为固定的 token 数/时间
- 一个 prompt 被分为多次调度
- 在 chunk 间隙处理高优先级的 decode 请求
- 每个 chunk 产生的 KV Cache 存储为 PagedAttention 格式

权衡： chunk 越小 $\rightarrow$ 调度延迟越低 $\rightarrow$ 但预填充总开销增加（更多的 kernel launch 开销和 KV 块管理开销）。

3.3　投机解码（Speculative Decoding）

3.3.1　核心思想

投机解码

自回归解码每次只生成 1 个 token（受限瓶颈）。投机解码用**草稿模型**（draft model）快速预测多个 token，再用**目标模型**（target model）一次性验证。

两步流程：

1. **草稿阶段：**草稿模型 M_q（~6B，比目标模型快 10×）自回归生成 γ 个 token（如 $\gamma = 5$）
2. **验证阶段：**目标模型 M_p（如 70B）一次前向验证这 γ 个 token
   - 如果第 i 个 token 被接受（$p_{\text{target}}(x_i) \geq p_{\text{draft}}(x_i)$）→ 输出该 token
   - 如果被拒绝 → 从修正分布重新采样，丢弃后续 token

关键保证：投机解码的输出分布严格等于目标模型的自回归分布（无偏）。这是数学上保证的。

3.3.2　接受/拒绝采样

Thrashing 修正采样

假定草稿 token x 以概率 $q(x)$ 生成，目标模型概率为 $p(x)$：

$$\text{接受}x \text{ 的概率} = \min\left(1, \frac{p(x)}{q(x)}\right) \tag{3.2}$$

若被拒绝，从 $\text{norm}(\max(0, p(x) - q(x)))$ 重新采样。

这保证了最终输出 token 的边缘分布为 $p(x)$。

接受率期望 $\approx 0.7 \sim 0.9$（取决于草稿模型质量），每次平均生成 $1 + \gamma \times$ 接受率 个 token，加速约 $2\times \sim 3\times$。

3.3.3　Medusa 与 Eagle

替代方案

Medusa：不单独训练草稿模型，而是在目标模型上附加多个额外的头（Medusa heads），每个头预测未来第 k 个 token。一次前向同时生成 K 个候选 token（树形结构），用树注意力（tree attention）验证。

Eagle：使用目标模型的倒数第二层特征 + 一个轻量级自回归头做草稿生成。比独立草稿模型更高效（特征复用）。

面试常见题

Q: 投机解码为什么理论是无偏的？

A: 因为接受/拒绝采样是标准的 Metropolis-Hastings 拒绝采样。验证阶段保证：输出 token 的分布恰好等于目标模型 $p(x)$ 的边缘分布。尽管使用了近似模型 $q(x)$ 做草稿，但验证阶段的修正采样完全纠正了偏差。

Q: 什么情况下投机解码加速效果最好？

A: (1) 草稿模型和目标模型分布接近（高接受率）；(2) γ 较大但草稿模型足够快；(3) 存在显著的草稿-目标模型速度差 $(10\times+)$；(4) 生成任务容易预测（如代码生成、翻译），而非创意写作。

Q: 与 batch decoding 的关系？

A: 互补。batch decoding 并行处理多个请求，投机解码加速单个请求的生成。两者结合可以进一步提升吞吐。

3.4 分离式部署（PD: Prefill-Decode Disaggregation）

分离式推理架构

将推理的预填充（Prefill）和解码（Decode）阶段分离到不同的 GPU 集群上运行：

- **Prefill 节点**：处理输入 prompt，产生第一个 token 和完整的 KV Cache。计算密集型（compute-bound）。
- **Decode 节点**：接收 KV Cache，逐 token 生成。显存带宽密集型（memory-bound）。

动机：

1. 预填充和解码的计算特性完全不同，混在一起难以优化调度
2. 预填充的延迟敏感度低（用户提交请求后可以等），解码的延迟敏感度高（流式输出）
3. 分离后可以独立扩缩容：预填充节点用 H100（高计算）、解码节点用 A100（大显存）

通信瓶颈：预填充节点需将 KV Cache 传输到 decode 节点。对于长序列，这可能成为瓶颈。使用 NVLink/RDMA 网络和 KV Cache 压缩（量化/稀疏化）缓解。

Mooncake / DistServe 等系统

目前主流的分离式推理系统：

- **DistServe (PKU)**：首次提出解耦预填充和解码的系统
- **Splitwise (MSR)**：将预填充和解码分离到两种 GPU 上
- **Mooncake (Moonshot/Kimi)**：大规模分离式部署，KV Cache 通过 RDMA 传输
- **Tetriserve**：将 prompt 分片到多个 decode 节点并行处理

3.5　大模型通信算子

3.5.1　张量并行（Tensor Parallelism, TP）

张量并行

在单层内将权重矩阵切分到多个 GPU 上，每个 GPU 计算一部分。

列切分：$\mathbf{W} \in \mathbb{R}^{d\times 4d}$ 沿列切分为 $[\mathbf{W}_1, \mathbf{W}_2]$（两个 GPU 各 $d \times 2d$）。输出 $\mathbf{Y} = [\mathbf{X}\mathbf{W}_1, \mathbf{X}\mathbf{W}_2]$。需要 all-gather 合并输出。

行切分：$\mathbf{W} \in \mathbb{R}^{4d\times d}$ 沿行切分为 $[\mathbf{W}_1^\top, \mathbf{W}_2^\top]^\top$。输入也需要切分 $\mathbf{X} = [\mathbf{X}_1, \mathbf{X}_2]$。输出 $\mathbf{Y} = \mathbf{X}_1\mathbf{W}_1 + \mathbf{X}_2\mathbf{W}_2$，需要 all-reduce。

通信：单层内至少 2 次 all-reduce（或 all-gather + reduce-scatter）。TP 度通常 ≤ 8（受限于机内 NVLink 带宽）。

3.5.2　流水线并行（Pipeline Parallelism, PP）

流水线并行

将模型层按顺序分配到不同 GPU 上：

- GPU0：层 0–9
- GPU1：层 10–19
- GPU2：层 20–29
- GPU3：层 30–39

每个 GPU 处理一个 micro-batch 后传给下一个 GPU。关键是减少流水线气泡：
1F1B (One-Forward-One-Backward)：交替执行前向和反向传播，减少空闲时间。
通信量：仅跨机器边界传递激活/梯度。通信量远小于 TP，但流水线气泡降低效率。

3.5.3 数据并行（Data Parallelism, DP）

数据并行

每个 GPU 持有完整模型副本，处理不同的 mini-batch。反向传播后，梯度通过 **all-reduce** 同步。
ZeRO (DeepSpeed)：对优化器状态、梯度和参数进行分区而非复制，显著减少显存：

- ZeRO-1：分区优化器状态（8× 显存节省）
- ZeRO-2：分区优化器状态 + 梯度
- ZeRO-3：分区优化器状态 + 梯度 + 参数（参数按需收集）

3.5.4 序列并行（Sequence Parallelism, SP）

序列并行

将序列长度维度切分到多个 GPU 上，每个 GPU 处理序列的一段。在非注意力操作（LayerNorm, FFN）中并行处理各自部分，在注意力操作前通过 all-gather 收集完整序列或使用 Ring Attention 在 GPU 间传递 KV。
Ring Attention：GPU 排列成环，KV 块在环上轮转，每个 GPU 交替做计算和通信。

面试常见题

Q: 3D 并行（TP+PP+DP）如何搭配？

A: 典型大规模训练配置：

- 同机 8 卡：TP（机内 NVLink 高带宽）
- 跨机：PP（跨机通信量小）
- 最外层：DP（可以异步，对通信延迟不敏感）

例如 1024 GPU 训练：TP=8, PP=16, DP=8。

Q: All-Reduce 的复杂度？

A: Ring All-Reduce 中，每个 GPU 的通信量为 $2(N-1)/N \times$ 数据量，步骤数为 $2(N-1)$。对于大梯度，带宽可以饱和。Tree All-Reduce 通信步骤数为 $2\log N$，但总通信量翻倍。

第四章　模型量化

4.1　量化基础

量化核心概念

将浮点权重/激活从 FP16 映射到低位宽整数（INT8, INT4, FP8）：

$$\tilde{\mathbf{W}} = \text{round}\left(\frac{\mathbf{W} - \text{zero}}{\text{scale}}\right) \tag{4.1}$$

$$\hat{\mathbf{W}} = \tilde{\mathbf{W}} \cdot \text{scale} + \text{zero} \tag{4.2}$$

量化粒度（精度与开销的权衡）：

- **每张量（per-tensor）**：一个 scale 对所有元素 → 精度最差
- **每通道（per-channel）**：每个输出通道一个 scale → 标准选择
- **每组（per-group）**：每 g 个元素一个 scale → GPTQ, AWQ 使用

对称量化 vs 非对称量化：

- 对称：$\text{zero} = 0$，范围 $[-127, 127]$
- 非对称：$\text{zero} \neq 0$，范围 $[0, 255]$。更好地利用范围，尤其对 ReLU 后的激活

4.2　GPTQ

GPTQ: 基于最优脑外科的量化

GPTQ 将 OBS（Optimal Brain Surgeon）剪枝算法适配到量化场景：
核心思想：

1. 逐列量化权重矩阵 $\mathbf{W}$ 的每一列
2. 量化一列后，通过 Hessian 信息将该列的量化误差补偿到其他未量化列上
3. 保证最终输出 $\mathbf{XW}$ 的误差最小化

算法流程：

Algorithm 3 GPTQ 量化（简化）

输入： 权重 $\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$, 校准数据 $\mathbf{X}$

计算 Hessian: $\mathbf{H} \leftarrow 2\mathbf{X}^\top \mathbf{X}$ (实际用 Cholesky 分解的逆)
for $i = 1$ **to** d_{in} **do**
  $\mathbf{w} \leftarrow \mathbf{W}[i, :]$ ▷ 第 i 行
  $\tilde{\mathbf{w}} \leftarrow \text{quant}(\mathbf{w})$ ▷ 量化
  $\boldsymbol{\delta} \leftarrow (\mathbf{w} - \tilde{\mathbf{w}})/[\mathbf{H}^{-1}]_{ii}$
  $\mathbf{W}[i+1:, :] \leftarrow \mathbf{W}[i+1:, :] + \mathbf{H}^{-1}[i+1:, i] \otimes \boldsymbol{\delta}$ ▷ 补偿误差

关键点：

- 需要校准数据（128 个样本，2048 tokens）
- 量化时间：7B 模型约 1 小时
- W4A16（权重 INT4，激活 FP16）：最常用配置
- 典型困惑度退化 < 1.0

4.3 AWQ

AWQ: Activation-Aware Weight Quantization

AWQ 基于一个关键发现：**权重并非同等重要**。大约 1% 的显著权重（对应大激活值的通道）贡献了大部分输出。

核心公式：

$$\mathbf{W}' = \text{quantize}(\mathbf{W} \cdot \text{diag}(\mathbf{s})) \cdot \text{diag}(\mathbf{s})^{-1} \tag{4.3}$$

其中 $\mathbf{s}$ 是每通道的缩放因子。通过缩放，将显著权重的量化难度" 转移" 到激活侧（而激活用 FP16，不量化）。

寻找最优 $\mathbf{s}$：

$$\mathbf{s}^* = \arg\min_{\mathbf{s}} \|\mathbf{W}\mathbf{X} - \mathbf{W}'\mathbf{X}\|_F^2 \tag{4.4}$$

通过网格搜索找到最优的 $\mathbf{s}$（固定搜索空间，效率高）。

与 GPTQ 对比：

- AWQ 不需要 Hessian 矩阵求逆（更快，更简单）
- AWQ 对校准数据的依赖更少
- GPTQ 理论上有更好的最优性保证

- 实践中两者效果接近，AWQ 实现更简单

4.4　量化对比与面试要点

表 4.1: 主流量化方案对比

方案	位宽	校准数据	速度	精度损失
GPTQ	W4A16	需要	中等	低
AWQ	W4A16	需要（少）	快	低
GGUF/llama.cpp	W2–W8	不需要	极快	中–高
bitsandbytes	W4/W8	不需要	快	中
SmoothQuant	W8A8	需要	快	极低
FP8 (native)	W8A8	不需要	极快	极低

面试常见题

Q: 为什么 Weight-Only 量化（W4A16）可行？

A: 大模型的权重矩阵通常有规律的分布（近似正态），而激活值分布不规则（有离群值）。权重量化到 4-bit 损失小，激活保持 16-bit 保证精度。此外，W4A16 的 dequantize 开销可以通过 kernel fusion 隐藏。

Q: 离群值（Outlier）对量化有什么影响？

A: 某些通道的激活值异常大（$6\sigma+$），会导致该通道量化的动态范围被”撑大”，其他正常值的量化精度显著下降。SmoothQuant 通过数学等效变换（$\mathbf{Y} = \mathbf{XW} = (\mathbf{X}\text{diag}(\mathbf{s})^{-1})(\text{diag}(\mathbf{s})\mathbf{W})$）将量化难度从激活侧迁移到权重侧。

Q: GPTQ 和 AWQ 的选择建议？

A: 需要最快量化 → AWQ；需要最优精度（含学术验证）→ GPTQ；CPU 推理 → GGUF；训练后立即量化且无校准数据 → bitsandbytes。

第五章　模型压缩与蒸馏

5.1　稀疏化

5.1.1　非结构化稀疏

非结构化稀疏

随机地将权重矩阵中的个别元素置零，通常通过 magnitude pruning：保留绝对值大的权重，将小的置零。

稀疏度： $s = \frac{\#\{\text{零权重}\}}{\#\{\text{总权重}\}}$

常见稀疏度 50%–90%。挑战：非结构化稀疏在 GPU 上不易加速（需要稀疏矩阵乘法支持，Ampere+ 支持 2:4 结构化稀疏）。

5.1.2　2:4 结构化稀疏

NVIDIA 2:4 稀疏

NVIDIA Ampere (A100) 引入的硬件加速稀疏格式：每 4 个连续元素中恰好保留 2 个（50% 稀疏）。

Tensor Core 支持： 直接加速，理论峰值翻倍。

训练方法： 训练时每隔一定步数做 magnitude pruning + 恢复训练（fine-tune）。如 ASP (Automatic SParsity) 或 SparseGPT。

5.1.3　半结构化稀疏（SparseGPT, Wanda）

SparseGPT

将 GPTQ 的逐列补偿思想应用于稀疏化，实现一次性（one-shot）50%+ 稀疏度的 LLM 剪枝。

Wanda (Weight and Activation)： 更简单的稀疏化方法，基于 $|W_{ij}| \cdot \|X_j\|_2$（权重绝对值 × 对应激活范数）排序，保留值大的。不需要 Hessian 求逆，速度极快。

5.2 蒸馏技术

5.2.1 知识蒸馏（Knowledge Distillation）

标准知识蒸馏

用教师模型（大、强）的输出分布指导学生模型（小、快）的训练。

软标签蒸馏：

$$\mathcal{L}_{\text{KD}} = \text{KL}\left(p^T(\cdot|x) \| p^S(\cdot|x)\right) \cdot T^2 \tag{5.1}$$

其中 T 是温度参数，p^T 是教师 softmax 输出。T^2 是梯度缩放因子。

硬标签 + 软标签：

$$\mathcal{L} = \alpha \cdot \mathcal{L}_{\text{CE}}(p^S, y_{\text{true}}) + (1-\alpha) \cdot \mathcal{L}_{\text{KD}} \tag{5.2}$$

5.2.2 LLM 特定蒸馏

LLM 蒸馏技术

1. 数据蒸馏（Data Distillation）：

- 用大型 LLM 生成高质量训练数据（如 Self-Instruct）
- 用小模型在这些数据上做 SFT
- 代表：Alpaca, Vicuna, Orca

2. Logit 蒸馏：直接匹配教师和学生的输出 logits（而非经过 softmax 的概率）。LLaMA 到 TinyLLaMA 常用。

3. 中间层蒸馏（Feature Distillation）：匹配中间隐藏层表示：

$$\mathcal{L}_{\text{feat}} = \|\mathbf{H}^T - \mathbf{H}^S \mathbf{W}_{\text{proj}}\|_F^2 \tag{5.3}$$

4. 逐步蒸馏（Step-by-Step Distillation）：教师模型不仅输出最终答案，还输出推理步骤（CoT）。学生模型学习” 模仿推理过程”。

5. 序列级蒸馏（Sequence-Level KD）：在 RL 框架下用教师模型的序列级评分（如 RLHF 中的奖励模型）指导学生。如 MiniLLM。

蒸馏 vs 量化 vs 剪枝选择指南

- **量化：**最快实施，保留所有参数但降低精度。适合已有模型快速部署
- **剪枝/稀疏化：**减少参数数量，但实际加速依赖硬件支持
- **蒸馏：**需要重新训练（或微调），但可以显著减小模型尺寸
- **实际中：**三者常结合。先蒸馏到中等模型，再量化为 INT4

5.3　多任务强化学习（MTRL）

MTRL for LLM

让 LLM 同时学习多个任务（翻译、摘要、问答等），通过 RL 优化一个联合目标：

多任务 RL 目标：

$$J(\theta) = \sum_{k=1}^{K} w_k \cdot \mathbb{E}_{\pi_\theta}\left[R_k(\tau)\right] - \beta \cdot \mathrm{KL}(\pi_\theta \| \pi_{\mathrm{ref}}) \tag{5.4}$$

其中 w_k 是任务 k 的权重，R_k 是任务特定的奖励函数。

挑战：

- **任务干扰**：任务 A 的梯度可能与任务 B 冲突
- **奖励尺度不一致**：摘要 ROUGE 分数 [0,1]，代码正确率 [0,100]
- **灾难性遗忘**：学习任务 C 后遗忘了任务 A

解决方案：

- **梯度投影（GradNorm, PCGrad）**：将冲突梯度投影到不冲突方向
- **弹性权重巩固（EWC）**：对重要参数加 L_2 惩罚
- **重放缓冲区（Experience Replay）**：混合存储各任务数据
- **任务特定适配器**：共享主干 + 任务特定 LoRA/Adapter

第六章　训练与对齐

6.1　SFT（监督微调）

SFT 详解

在高质量的（指令, 回答）对上用标准交叉熵损失微调预训练 LLM：

$$\mathcal{L}_{\text{SFT}} = -\sum_{t=1}^{T} \log p_\theta(y_t | y_{<t}, x) \tag{6.1}$$

数据格式（以对话为例）：

```
### Human: 请解释量子纠缠是什么？
### Assistant: 量子纠缠是一种量子力学现象...
```

或使用 ChatML / ShareGPT 格式。

SFT 的关键要素：

- **数据质量 > 数据数量**：LIMA 论文证明，1000 条高质量 SFT 数据即可产生强指令遵循能力
- **数据多样性**：覆盖代码、数学、创意写作、推理等
- **损失掩码**：只对 Assistant 部分计算损失，Human 部分 mask 掉
- **学习率**：通常 $10^{-5} \sim 10^{-6}$，训练 1–3 epochs

6.2 RLHF（基于人类反馈的强化学习）

6.2.1 三阶段流程

RLHF 完整流程

阶段 1: **SFT**：在高质量指令数据上微调基础模型

阶段 2: **奖励模型（RM）训练**：

- 对同一 prompt，SFT 模型生成多个回答
- 人类标注者对回答排序
- 用 Bradley-Terry 模型训练奖励模型

阶段 3: **PPO 微调**：用 RM 作为奖励函数，用 PPO 优化策略

6.2.2 奖励模型训练

Bradley-Terry 偏好模型

给定 prompt x，回答 y_1 优于 y_2（记为 $y_1 \succ y_2$），偏好概率为：

$$P(y_1 \succ y_2|x) = \frac{\exp(r_\phi(x, y_1))}{\exp(r_\phi(x, y_1)) + \exp(r_\phi(x, y_2))} = \sigma(r_\phi(x, y_1) - r_\phi(x, y_2)) \tag{6.2}$$

损失函数：

$$\mathcal{L}_{\text{RM}} = -\mathbb{E}_{(x,y_w,y_l)} \left[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))\right] \tag{6.3}$$

其中 y_w 是" 赢" 的回答，y_l 是" 输" 的回答。r_ϕ 通常是在 SFT 模型上加一个线性头（最后 token 的隐藏状态 $\to$ 标量奖励）。

6.2.3 PPO 微调

PPO for RLHF

优化目标（含 KL 惩罚）：

$$J(\theta) = \mathbb{E}_{x\sim\mathcal{D}, y\sim\pi_\theta} \left[r_\phi(x, y) - \beta \cdot \text{KL}(\pi_\theta \| \pi_{\text{ref}})\right] \tag{6.4}$$

其中 π_{ref} 是 SFT 模型（冻结），β 控制策略偏离程度。

PPO 更新步骤：

1. 用策略 $\pi_{\theta_{\text{old}}}$ 采样一批回答
2. 用奖励模型 r_ϕ 对每个回答打分
3. 计算优势函数（GAE）

4. 用裁剪目标更新 θ（K 个 epoch）

5. 每轮更新后检查 KL 散度，必要时提前停止

PPO 四个模型：

- **策略模型** π_θ：被优化的模型（需要梯度）
- **参考模型** π_{ref}：SFT 模型（冻结，计算 KL）
- **奖励模型** r_ϕ：评估回答质量（冻结）
- **价值模型** V_ψ：估计期望回报（需要梯度）

6.3　偏好学习（DPO 等）

6.3.1　DPO（Direct Preference Optimization）

DPO 核心公式

DPO 将 PPO 的两阶段（先训 RM 再训策略）合并为一步，直接在偏好数据上优化策略：

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x,y_w,y_l)}\left[\log\sigma\left(\beta\cdot\log\frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta\cdot\log\frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)\right] \tag{6.5}$$

推导（面试重点）：

从 Bradley-Terry 模型出发：$p(y_w \succ y_l) = \sigma(r(y_w) - r(y_l))$

在 RLHF 的 PPO 目标中，最优策略 π^* 满足：

$$\pi^*(y|x) = \frac{1}{Z(x)}\pi_{\text{ref}}(y|x)\cdot\exp\left(\frac{1}{\beta}r(x,y)\right) \tag{6.6}$$

反解出：$r(x,y) = \beta\cdot\log\frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta\log Z(x)$

代入 Bradley-Terry：

$$\begin{aligned} p(y_w \succ y_l) &= \sigma(r(y_w) - r(y_l)) \\ &= \sigma\left(\beta\cdot\log\frac{\pi^*(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta\cdot\log\frac{\pi^*(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right) \end{aligned}$$

其中 $Z(x)$ 项被消去。最后取负对数似然即得 DPO 损失。

6.3.2 DPO 变体

偏好学习方法谱系

- **IPO (Identity Preference Optimization)**：用平方损失替代交叉熵，解决 DPO 在某些情况下过度拟合的问题
- **KTO (Kahneman-Tversky Optimization)**：只需二元反馈（好/坏），不需配对比较
- **ORPO (Odds Ratio PO)**：将 SFT 和偏好学习合并到一个损失中
- **SimPO**：直接用序列长度归一化的平均 log 概率作为隐式奖励
- **R-DPO / Step-DPO**：针对推理/数学场景的 DPO 变体，对推理链进行偏好学习

面试常见题

Q: RLHF vs DPO，如何选择？

A:

- **RLHF**：需要训练 + 维护 RM，工程复杂度高（需 4 个模型同时在 GPU 上）。但在线数据收集 + PPO 探索可以探索到超出偏好的更优策略。适合资源充足的大型团队。
- **DPO**：简洁，只需 SFT 模型 + 参考模型（可同一模型）。直接优化偏好数据，无需训练 RM。但只能利用已有的离线偏好数据，无法在线探索。
- **实际趋势**：DPO 在小规模/研究中占主导；大厂（OpenAI, Anthropic）仍用 RLHF。也有迭代 DPO（iterative DPO）尝试结合两者优势。

Q: DPO 的参考模型 π_{ref} 为什么重要？

A: 防止策略”讨好”偏好数据。如果没有 KL 约束（即无 π_{ref} 项），DPO 等价于在偏好对上做最大化对数似然，策略可能退化——输出高奖励但离谱的文本（reward hacking）。π_{ref} 通过 KL 惩罚将策略锚定在原始分布附近。

Q: DPO 数学推导中 $Z(x)$ 为何消失？

A: $Z(x) = \sum_y \pi_{\text{ref}}(y|x) \exp(r(x,y)/\beta)$ 是分区函数，只依赖于 x。在对 $r(y_w) - r(y_l)$ 做差时，$Z(x)$ 项对消——这是 DPO 数学上的核心技巧。

6.4　模型对齐全景

对齐技术对比

表 6.1: 对齐方法对比

方法	需要 RM	在线/离线	复杂度	适用阶段
SFT	否	离线	低	第一阶段
RLHF (PPO)	是	在线	极高	第三阶段
DPO	否	离线	低	替代 RLHF
KTO	否	离线	低	仅有二元反馈
RLVR (RL with Verifiable Rewards)	否 (规则奖励)	在线	高	数学/代码
Constitutional AI	是	混合	高	持续对齐

其中 **RLVR** 使用可验证的规则奖励（如代码是否正确通过测试、数学答案是否匹配），不需要人工标注的偏好数据，是近期热点。

第七章　能力专题

7.1　Planning（规划）

LLM 的规划能力

定义：给定目标，自动分解为子任务并按正确顺序执行。

方法：

- **Plan-and-Solve：**先生成完整计划，再逐步执行。提示模板：”Let’s first understand the problem and devise a plan...”
- **ReAct：**交替执行” 推理（Thought）” 和” 行动（Action）”。在环境中执行行动后观察结果，再推理下一步。
- **Tree-of-Thoughts (ToT)：**在每一步维护多个候选计划，用 BFS/DFS 搜索，用 LLM 评估每个候选。
- **LLM+P / LLM+PDDL：**用 LLM 将自然语言目标转为 PDDL（规划语言），用经典规划器求解，转回自然语言。
- **Code-as-Planning：**让 LLM 生成可执行代码来实现规划（如机器人控制代码）。

当前局限：LLM 的规划能力在长程、复杂规划任务上仍显著弱于专门规划器。主要因为 token 级自回归生成缺乏全局回溯机制。

7.2　Reasoning（推理）

推理增强技术

Chain-of-Thought (CoT)：

- 在 prompt 中加入”Let’s think step by step”
- 引导模型输出中间推理步骤

- 在 GSM8K 上将 GPT-3 175B 从 17% 提升到 58%

Self-Consistency：对同一问题多次采样 CoT 路径，取出现最频繁的最终答案（多数投票）。
Least-to-Most：将问题分解为从简单到复杂的子问题序列，逐一解决。
验证器（Verifier）：训练一个独立的验证模型评估推理步骤的正确性，选择得分最高的候选。
o1 / DeepSeek-R1 风格：在 RL 训练中强化长推理链，模型自发涌现"Aha moment"（自我修正/反思）。
内部推理：模型在输出最终答案前产生大量内部 token（latent reasoning），这些 token 不对用户展示。如 Coconut (Meta)。

7.3 Coding（代码生成）

代码生成核心技术

1. 执行反馈：

- 生成代码 → 在沙箱执行 → 获取测试结果/错误信息
- 将错误反馈拼接到上下文，让模型自我修复
- 多轮迭代直到通过所有测试

2. 代码特定 RL：

- 奖励 = 编译通过 + 测试通过率
- 用 PPO/DPO 在代码数据上微调
- RFT (Rejection Fine-Tuning)：只保留通过的代码做 SFT

3. 搜索增强代码生成：

- AlphaCode/CodeContests：采样 10^6 个候选，过滤 + 聚类 + 重排序
- 结合 MCTS 搜索最优代码路径
- 结合测试时计算量提升（test-time compute scaling）

4. Agentic Coding (SWE-bench)：

- LLM 作为自主代码智能体
- 能读仓库、定位 bug、写 patch、运行测试
- 使用工具：文件系统、git、linter、测试框架

7.4 Instruction Following

指令遵循

训练数据构造：

- Self-Instruct：用强模型生成指令-回答对
- Evol-Instruct：通过规则变体（加约束、深化、增加推理步骤）演化出更复杂的指令
- Backtranslation：对任意文本反向生成对应指令

IF-Eval（评估指标）：

- 检查输出是否遵循特定格式约束（如” 用 JSON 回答”、” 不少于 100 字”）

- 严格匹配 vs 松散匹配
- 约束类型：长度、格式、内容、风格、关键词

长度偏差问题：DPO/RLHF 训练容易导致模型输出越来越长（" 奖励篡改")。解决方案：

- 在奖励中添加长度惩罚
- R-DPO：对过长回答额外扣分
- 归一化序列概率

7.5 Hallucination Control

幻觉控制方法

1. RAG（检索增强生成）：

- 先从知识库检索相关文档
- 将文档拼接到上下文
- 模型基于检索结果生成回答
- 显著减少事实性幻觉（但不能完全消除来源幻觉）

2. 后验推理/验证：

- SelfCheckGPT：多次采样，检测回答间的一致性
- FactScore：将回答分解为原子事实，逐一验证
- 训练幻觉检测分类器

3. 训练侧方法：

- RLHF/DPO 训练中对幻觉标记负奖励
- 让模型学会说" 我不知道"
- 不确定性量化：模型不仅输出答案，还输出置信度分数

4. 推理时干预：

- 对比层解码（DoLa）：对比后期层和早期层的 logits 差异，增强事实性输出
- 上下文感知解码（CAD）：对比有/无外部知识的输出，增强忠实度

7.6　Context Learning（上下文学习）

In-Context Learning (ICL)

本质：LLM 无需参数更新，通过在 prompt 中提供少量示例来适应新任务。
工作机制（理论解释）：

- **隐含贝叶斯推断**：ICL 可视为 LLM 在上下文示例上做隐式贝叶斯参数估计
- **梯度下降等价性**：Transformer 的注意力机制在数学上可模拟一步梯度下降
- **任务向量**：ICL 示例在模型的表示空间中形成了一个隐式的" 任务向量"

长上下文学习：

- 将数千个示例直接放入 prompt（Many-Shot ICL）
- Gemini 1.5 Pro 支持 10M token 上下文
- 需要高效的注意力机制（FlashAttention, Ring Attention）
- 长上下文中存在” 迷失在中间” 问题（Lost in the Middle）

检索增强 ICL：从大量示例中检索与当前输入最相似的 k 个示例放入 prompt。

第八章　系统与工程

8.1　MFU（模型浮点运算利用率）

MFU 定义与计算

定义：实际达到的 FLOPs 与 GPU 理论峰值 FLOPs 的比值。

$$\text{MFU} = \frac{\text{实际 FLOPs}}{\text{理论峰值 FLOPs}} \tag{8.1}$$

实际 FLOPs 估计（Transformer 前向传播）：对每个 token（含 KV Cache 的 decode 阶段）：

$$\text{FLOPs} \approx 2 \times (n_{\text{params}} + 2 \times n_{\text{layers}} \times n_{\text{ctx}} \times d_{\text{model}}) \tag{8.2}$$

其中第一项来自权重矩阵乘法，第二项来自注意力计算。

典型 MFU 值：

- 大模型预训练：50%–60%（Megatron-LM 优化后）
- LoRA 微调：30%–45%
- 推理（decode 阶段）：1%–5%（memory-bound，算力大量空闲）
- 推理（prefill 阶段）：40%–60%

低 MFU 的主要原因：

- GPU 等待 HBM 数据（显存带宽瓶颈）
- 内核启动开销（kernel launch overhead）
- 负载不均衡（某些 GPU 等待其他 GPU）
- 通信开销（TP/PP 间的同步）

8.2 后训练优化（Post-Training）

后训练优化技术

1. 量化感知训练（QAT）：在前向传播中模拟量化操作，反向传播中用 STE（Straight-Through Estimator）传递梯度。

- 比 PTQ 精度高，但需要重新训练
- LLM-QAT：对 LLM 做 QAT（但成本极高，不常用）

2. 剪枝 + 恢复训练：剪枝 50% 参数后，对剩余参数微调 100–1000 步恢复精度。

3. 蒸馏 + 微调：先用大模型蒸馏，再用 SFT/DPO 微调蒸馏后的小模型。

4. 连续预训练（CPT）：在领域数据（如医疗、法律、代码）上继续预训练，打乱分布。

5. Post-Training Quantization (PTQ)：GPTQ, AWQ, SmoothQuant 等，不更新参数直接量化。

8.3 总结：大模型技术栈全景图

技术栈层次

L1: **硬件层：**GPU (H100, A100), TPU, 互联 (NVLink, InfiniBand)

L2: **算子层：**FlashAttention, cuBLAS, NCCL, CUDA Graph

L3: **并行层：**TP, PP, DP, SP, ZeRO, FSDP

L4: **架构层：**Transformer, MoE, Mamba, GQA, RoPE

L5: **训练层：**预训练, SFT, RLHF, DPO, 课程学习

L6: **推理层：**vLLM, TensorRT-LLM, 量化, KV Cache, 投机解码

L7: **应用层：**RAG, Agent, Tool Use, Planning, Reasoning

大模型面试高频题汇总

1. FlashAttention 的分块 softmax 如何保证数值稳定性？
2. vLLM PagedAttention 的块表设计细节？
3. MoE 的负载均衡损失为什么是 $N \sum f_i P_i$ 的形式？
4. KV Cache 的显存计算方法？

5. 投机解码的接受/拒绝采样如何保证无偏？
6. 3D 并行中 TP、PP、DP 如何选择比例？
7. GPTQ 中 Hessian 的作用是什么？
8. AWQ 的缩放因子 $\mathbf{s}$ 如何找到？
9. DPO 数学推导中 $Z(x)$ 为什么能消去？
10. RLHF 和 DPO 的核心区别及各自的适用场景？
11. 长上下文中的”Lost in the Middle” 现象及解决方案？
12. MFU 低的主要原因有哪些？
13. 模型融合中 TIES-Merging 的三个步骤？
14. RAG 和长上下文的关系与选择？
15. 量化中的离群值如何处理？